

ShellVis: Sandboxed Live Programming for Shell Scripts

Joshua Horowitz
joshuah@alum.mit.edu
University of Washington
Seattle, United States

Jeffrey Heer
jheer@uw.edu
University of Washington
Seattle, United States

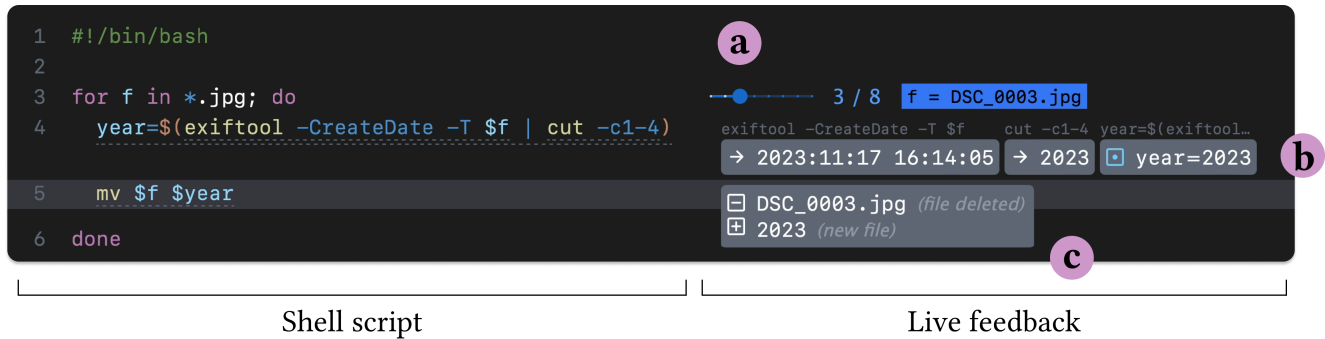


Figure 1: A ShellVis user writes a script to sort photos into folders by year. The script’s source is shown on the left. On the right, focus+context visualizations convey script outputs. A loop slider (a) indexes iterations of the for loop, while abbreviated bubbles (b) show command output and effects on shell variables. An expanded bubble for the current focus (c) shows file system effects. The script contains a critical mistake, but ShellVis ran it in a sandbox so the user’s photos are unharmed.

ABSTRACT

Live programming provides visibility to programmers by running and tracing programs as they are edited. However, for programs with potentially harmful side effects, liveness can turn mistakes into disasters. We propose enabling live programming in environments with side effects via sandboxing: confining effects to a simulation of the true environment. We apply *sandboxed live programming* in the challenging context of shell scripting: a ubiquitous and powerful—yet notoriously opaque and error-prone—tool. *ShellVis* provides line-by-line feedback on a shell script’s run-time behavior, with file operations sandboxed via a safe overlay of the file system. A qualitative user evaluation finds ShellVis to be helpful to participants, replacing tedious existing practices and instilling confidence. Participant responses also reveal areas for future research, particularly bridging the gulf of execution alongside the gulf of evaluation. ShellVis serves as a case study of how sandboxing can bring live-programming techniques into the many real-world programming contexts where side effects are important.

KEYWORDS

live programming, shell scripts, sandboxing

1 INTRODUCTION

While computer use has generally moved from the command line to GUIs, the shell remains a essential site of activity for users who find value in its precision and power. The shell is also a gateway guiding end users into programming [42]. Everything a user types at a command line is written in a programming language like bash [12]. By packaging a list of commands into a text-file program, a *shell script*, a user can document a process and make it re-usable. Starting with this straight-line automation, further levels of programmability

open up: parameterizing scripts with command-line arguments, iterating through files with for loops, and so on.

Shell scripts have a much broader reach, both actual and potential, than just system administrators and software engineers. For instance, shell scripts have been adopted by many practitioners that work with data, including data journalists [38], scientists [57], and sociologists [21]. Indeed, since nearly all computer users work with files and programs—the main characters of shell scripts—scripts have the potential to be useful to nearly everyone. We envision a world where shell scripts are approachable tools, amplifying the power of diverse computer users.

In practice, the picture is not so rosy. Many obstacles lie in the way of diverse users wielding shell script super-powers. While these include the learning challenges shared by all programming languages, shell scripts present unique challenges which make them “powerful, ubiquitous, and reviled” [18] even among experienced programmers. These include idiosyncratic syntax and unfamiliar semantics, in a “fast-paced high-stakes” environment where “a single typo could erase entire hard drives” [18].

We hypothesize that these downsides can be significantly mitigated with an interactive programming environment. In particular, we draw upon the techniques of *live programming* [52] – running a program as the programmer edits it and providing detailed feedback on what their in-progress program does. With live programming, a programmer can see the actual data running through their program and plan their next step in response. They can quickly detect problems and survey program behavior to diagnose root causes.

While live programming has proven successful in contexts like spreadsheets and computational notebooks, it faces new challenges when applied to shell scripts. Shell scripts are typically written not (only) to compute numbers or make graphs, but to perform *side effects*, such as modifying files, which are typically difficult to undo.

How can liveness, which relies upon running a program over and over again as it is modified, be made to work in an environment where running the program might be dangerous?

To explore these questions, we contribute **ShellVis**, a live programming system for bash shell scripts made possible through *sandboxing*. As a script is edited in a conventional text editor, ShellVis runs it, presenting the programmer with a fine-grained, navigable display of its run-time behavior, including text written to output streams, changes to shell variables, and modifications to the file system. To make sure every run of the program starts from the same “clean slate”, and to protect the user from irreversible mistakes, ShellVis runs the script in a virtual file system which simulates changes to the user’s real file system without actually modifying it.

Our goals with ShellVis are to help programmers understand programs (both their own & others’) and write programs more effectively. To assess this, we ran a user study in which seven participants with 1-20 years of shell experience tried out ShellVis for the first time, performing four shell-scripting tasks. Results were encouraging, with participants unanimously expressing that ShellVis was helpful, comfortable to use, and desirable for their own work. From participants, we learned how the feedback & sandboxing provided by ShellVis could replace tedious existing practices. Critically, participants reported that ShellVis made them feel safer and more confident as they worked with shell scripts, a quality we believe is especially important in end-user-programming and learning contexts. Finally, we identified ways that ShellVis does not yet meet users’ needs, from helping them figure out what code to write in the first place, to sandboxing more kinds of effects, to better integrating live feedback with existing editors.

In summary, this paper makes the following contributions:

- Identifying *sandboxed live programming* as a way to provide fine-grained feedback to programmers working in contexts, like shell scripting, centered around side effects.
- ShellVis, a prototype sandboxed live-programming environment for shell-scripting that uses overlay file systems and visualization techniques to provide safe, fine-grained feedback to programmers.
- A user evaluation of ShellVis demonstrating that programmers found the combination of live-programming and sandboxing to be helpful, while revealing both the efficacy of ShellVis’s approach and needed future directions for sandboxed live programming to reach full effectiveness.

Together, these contributions suggest there is untapped potential for fine-grained live programming to support real-world programming contexts, if side-effects can be appropriately sandboxed.

2 BACKGROUND AND RELATED WORK

2.1 Shell Scripts

Unix-based operating systems (including Linux and Mac OS) provide a command interpreter called the *shell* [53, 54]. The shell lets users perform one-off commands to start and stop processes, manipulate the file system, and more. On top of these atomic operations, the shell provides ways to compose operations together: the celebrated inter-process pipe [28, 54] as well as basic programming constructs like variables and loops. With these scripting constructs, the shell becomes a tool for automation.

On GUI-based computers, graphical mechanisms have largely replaced the shell: users launch applications from icons and drag files between windows representing directories. But even in this setting, the shell maintains unique advantages, including scriptability. While a user can drag a single file in a file manager, they cannot record this action for later re-use, or generalize it to a broader pattern to run in a loop. Graphical automation is an active field of research, and can be found in modern OSes in places like Shortcuts [1], but many users still reach for the shell for its precise control and its established ecosystem of command-line tools.

Unfortunately, “[s]hell scripts are hard even for experts” [19]. We have been unable to find formal studies in the literature specifically focused on challenges with shell scripts, but general dissatisfaction with the shell is broadly acknowledged. Greenberg et al. [17] do not hesitate to call it “a confusing and dangerous object of disgust.” Participants in their HotOS 2021 panel [19] proposed a number of contributory factors, including opaque shell syntax, a lack of visibility into shell state, and dangerous, irreversible operations. Testimony from participants in our own study support these hypotheses (§8.1). ShellVis seeks to address these obstacles through *sandboxed live programming*.

2.2 Programming Environments

It is helpful to position ShellVis as a *within-code live-programming environment*, and to place this category in relation to the neighboring concepts of *REPL*, *notebook*, and *debugger*.

The Unix command line is a REPL (read-eval-print loop) for the shell language. REPLs provide fast feedback, enabling experimentation and iterative development. However, going from running one-off commands in a REPL to building a durable program is not always easy, and REPLs in many other languages have been supplanted in practice. For instance, while Python can be run in a REPL, interactive use of Python has shifted strongly towards *notebooks* like Jupyter [33]. A notebook can reproduce the append-only pattern of a REPL, but it doesn’t need to. Notebook cells are mutable, and can be revised and re-run. In this way, notebooks combine the document-like persistence of a text-file script with the immediacy and visibility of a REPL. The success of notebooks raises the question (briefly alluded to by Greenberg et al. [18]) as to whether shell scripts could similarly benefit from a hybrid interface, partway between REPL and traditional script editor.

ShellVis is such a hybrid interface. Rather than taking the form of a notebook, it takes the form of a *within-code live-programming environment* (WCE): an environment that “augments [a] textual-code editor... with in-context displays showing run-time behavior” [26]. While notebooks achieve liveness with a cell-based structure, WCEs like Projection Boxes [34] provide live feedback within a traditional textual programming environment.¹

WCEs have some advantages over notebooks. They co-exist seamlessly with the corpus and ecosystem of a language. Existing programs can be loaded into a WCE and existing tools can be used alongside it. As a text-based environment, a WCE benefits from broad support for plain text, from powerful editors to revision control systems like git. Finally, WCEs can bring live feedback “deeper” into a program than is afforded by a notebook, such as into

¹For more examples see Horowitz & Heer [26].

loop and function bodies [25]. We chose to build ShellVis as a WCE for these advantages.

Like WCEs, *debuggers* provide visibility into run-time behavior. However, they are designed for a fundamentally different purpose. While debuggers are used to investigate a program after something has gone wrong, WCEs provide feedback to support writing a program in the first place. Debuggers rarely support this latter task. Traditional single-step debuggers only provide delicate, keyhole visibility into a program, rather than always-on feedback. Omniscient debuggers which “make it possible to navigate backwards in time within a program execution trace” [50] are closer in design and spirit to WCEs, but are still not designed to provide feedback during iterative program development.

2.3 Live Programming with Side Effects

Live programming relies on re-running programs (or parts thereof) in response to edits so programmers can see the effects of their edits. For this reason, live programming has historically been most at home in pure functional environments like spreadsheets, where re-runs are safe and predictable.²

What happens when we remove this constraint and make imperative systems live? Numerous live environments (including Projection Boxes [34] and Seymour [31]) have provided visibility into programs that mutate local state: state contained *within* a program which can be cleanly re-initialized on each execution. ShellVis takes on a larger challenge: creating a live programming experience for shell scripts that interact with the outside world through *side effects*.

There are two reasons why side effects cause problems for live programming. First, the execution of a script in a live-programming environment is intended to be *speculative*, showing what the program *would* do if run in a particular starting world, so the programmer can iterate until the program does what they want. If this program caused real side effects as it re-ran, the programmer would be working on constantly shifting ground. We need to make programs with side effects *repeatable*. Secondly, by moving to a regime where a program is executed automatically and repeatedly as it is edited rather than at deliberately chosen points, live programming increases the chance that a side effect will cause a disastrous, irreversible result. We need to make programs with side effects *safe*.

The need for live programming to grapple with external side-effects has recently been recognized by Sulir et al. [55], who concluded from their study of Java source code that “neglecting I/O is not a viable option for tool designers”. Yet, “studies about this phenomenon are rare” and “I/O is rarely taken into account when designing [live programming] tools.” In order to make a reactive, live programming experience possible, ShellVis takes the approach of sandboxing access to the file system. We are not aware of any other live-programming projects that sandbox or otherwise contain their external effects, as ShellVis does.

²Pure functional programming is based on producing new values from old values without modifying (“mutating”) values in place. Spreadsheet formulas follow the pure functional discipline. Pure functional programs are easy to re-run; running the same program with the same inputs produces the same outputs. As an example of how pure functional architecture benefits liveness: Cells in Jupyter notebooks cannot be reliably re-run in response to upstream changes because information flows between cells via hard-to-track mutations. In contrast, *reactive* notebooks like Observable follow the spreadsheet execution model [45], producing a more live experience.

2.4 Tools for Shell Scripting

The tool most relevant to this work is `try`, a command-line program for Linux which “lets you run a command and inspect its effects before changing your live system” [56]. To use `try`, a user takes a command they want to run at the command line and prepends `try` to it. `try` then runs the target command in a partial sandbox which simulates writes to the file system without actually performing them. It reports to the user what files have been modified by the command, and offers the option of “committing” the changes.

Sandboxing in ShellVis is directly inspired by `try`. Specifically, we borrow `try`’s use of an “overlay file system” to protect the original file system and detect file system modifications to report to the user (§6.1). ShellVis’s unique insight is to employ sandboxing in the context of a larger live programming environment. ShellVis provides not just a top-level summary of what a script does to the file system (as `try` does), but fine-grained information on what individual commands in a script do. ShellVis complements information on file system modifications with other streams of information, including command output and changes to shell variables (§5).

Live programming tools for authoring shell pipelines include Ultimate Plumber [8] and Pipecut [37]. ShellVis generalizes these tools, visualizing not just pipelines but entire scripts. ShellVis also sandboxes side effects, while these prior tools do not.

Several debuggers exist for shell languages. BASHDB is a single-step debugger for Bash [3]. The Shtepper is a research program stepper for POSIX-compatible shell scripts built on the Smoosh executable semantics [16]. Farther from Unix, Windows PowerShell has traditional single-step debugging [40]. Shell debuggers do not seem to be used much, at least with the Unix shell. For visibility and debugging, the tool of choice is `echo`, the shell’s “print” statement. Some programmers may also use features like `set -x` and `set -v` which tell the shell to log all commands as they run.

Many other tools are available for working around the quirks and corner cases of the shell. ShellCheck is a popular static analysis tool that detects dangerous patterns in scripts [24]. NoFAQ suggests fixes to shell commands based on error messages [9]. explainshell breaks complex shell commands into pieces and provides documentation from man pages [29]. Finally, LLM-powered code assistants have been used for shell scripting as they have for every other domain of programming. We consider all these tools to be complementary to ShellVis, and discuss code assistants in particular more in §9.1.

2.5 Summary

Tools providing visibility into the run-time behavior of shell scripts are rare and inadequate. ShellVis fills this gap with a *within-code live programming environment* which shows the user as much information as possible about the behavior of an under-development script. A key enabling technology for this is *sandboxing* the script’s effects to make them repeatable and safe. While sandboxing shell scripts has been explored before in `try`, sandboxing has not yet been combined with fine-grained feedback. We believe this combination provides a powerful and unexplored synergy.

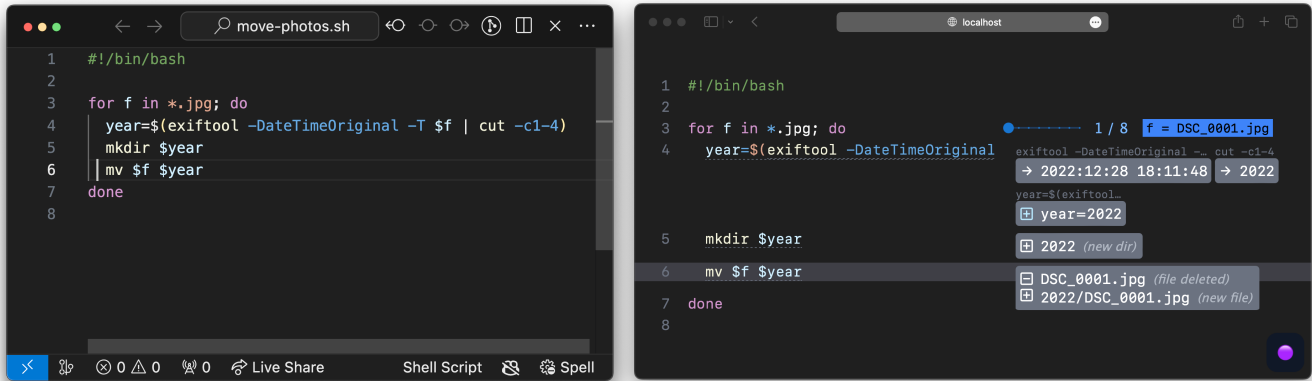


Figure 2: ShellVis in use on a user’s desktop. A text editor (Visual Studio Code) is open on the left and the ShellVis visualizer is open in a web browser on the right. The visualizer’s focus is linked to the text cursor position in the editor.

3 SANDBOXED LIVE PROGRAMMING IN SHELLVIS

Like most “within-code live-programming environments” (e.g., Projection Boxes [34]), ShellVis uses a four-step feedback loop:

- (1) **Editing:** Provide the user with a means to edit a program.
- (2) **Running:** At some cadence, automatic or user-defined, run the program.
- (3) **Tracing:** Monitor the program’s execution to extract fine-grained information.
- (4) **Viewing:** Report tracing information to the user in a helpful way.

Before jumping into a demonstration, we provide a quick overview of how ShellVis approaches these four steps.

Editing: ShellVis lets users edit code in the text editor of their choice rather than requiring that they move to a custom editor. Existing editors are polished, familiar, and entrenched. Any tool demanding programmers leave behind their favorite text editor faces an uphill battle. We also chose this path simply to focus on our core objective: the provision of run-time feedback.

Running: The running stage is where ShellVis introduces the essential component of *sandboxing*. Shell scripts can have many different effects, including file system access, network access, interaction with other processes (including launching or killing processes), access to devices, and assorted system calls. We chose in this project to focus on sandboxing file system access.

The file system is a natural starting point for sandboxing. It is a central site of action and coordination in modern operating systems. Explicit file management is a primary use-case for shell scripts, and the file system is used as backing state for many infrastructures (like git). We leave sandboxing of other effects to future work, but we discuss a broad range of ways they might be handled in §9.3.

Tracing: The machinery ShellVis uses to sandbox file system access can also be leveraged to tell us what effects each command in a script has on the file system: what files are added, removed, or modified. This tracing of external effects provides essential information to programmers to make sure their scripts are doing what they expect. We augment this with additional trace information on

process output, as well as changes in shell state such as variables and working directory.

Viewing: Presenting the large amount of traced information to the user in a helpful way is a significant challenge. ShellVis manages this complexity with a focus+context “fisheye view” [13, 14] that adjusts the level of detail based on the user’s current focus, and with interactions for surveying loop iterations.

4 USAGE SCENARIO

Figure 2 shows the layout of a user’s screen while using ShellVis. A text editor (Visual Studio Code) is open on the left, displaying a finished shell script. On the right, the ShellVis visualizer is open inside a web browser. It displays the same script, together with output annotations inside gray “bubbles.” As the user edits the script on the left, the display on the right updates.

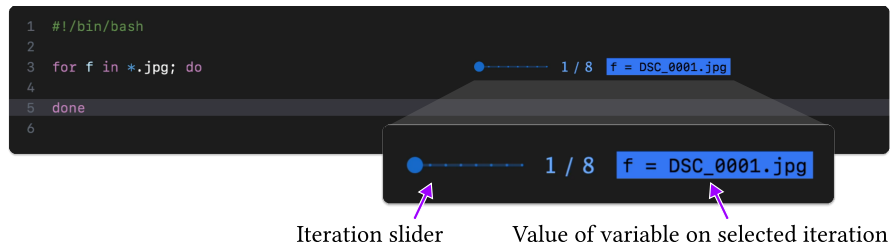
The purpose of the script shown is to sort a directory of JPEG photos into subdirectories based on the year each photo was taken. Let’s examine how ShellVis assists the user through the steps of making this script, with Figure 3 as a visual aid.

Step 1: Looping through files. The user wants to perform an action for each JPEG image, so they start with a `for` loop. They type out a loop with an empty body and press `Ctrl-S` to save. When ShellVis detects that the script has changed, it automatically re-runs it and displays a visualization of its execution to the user. In this case, a slider appears to the right of the line with `for`, reporting that the loop has run 8 times, and on its first iteration the variable `f` has taken on the value “DSC_0001.jpg”. Scrubbing the slider left and right, the user can see what other values this variable takes on. This is the sort of information that might ordinarily be obtained by adding an `echo $f` in the loop, but ShellVis displays information like this automatically and in-context. Once the loop’s body is no longer empty, this slider will also control which iteration is used as context for visualization of the body’s execution.

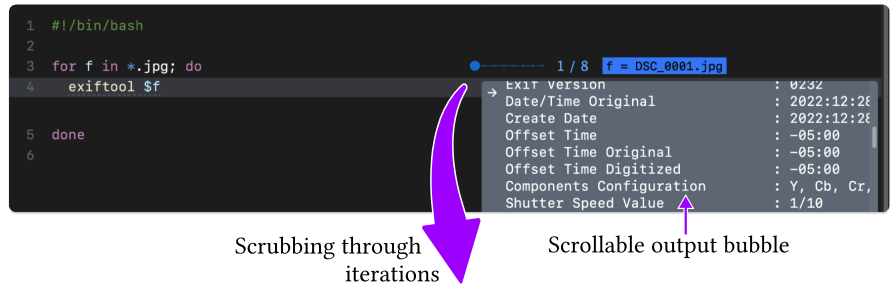
Step 2: Inspecting files. Now the user wants to figure out what year these photos were taken. They add a call to `exiftool $f`, which prints out the file’s EXIF metadata. This output is displayed in a standard-output bubble. In this case, the output is large and the

Step 1: Looping through files

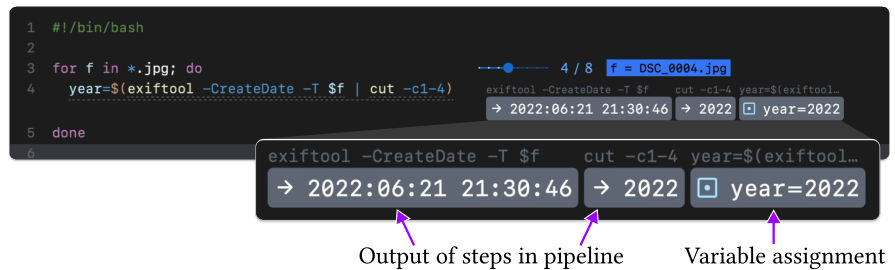
When the user adds a **for** loop, a slider appears for scrubbing through iterations.

**Step 2: Inspecting files**

Output from the call to **exiftool** is shown in a bubble on the right-hand side.

**Step 3: Pipe & variable**

Output is piped into a second command and assigned to a variable. Bubbles appear for each step and the assignment.

**Step 4: A mistaken move**

The bubble for **mv** shows that the image files have been moved, mistakenly, to files named after years.

**Step 5: Fixing the mistake**

The user adds a call to **mkdir** and confirms that this fixes the call to **mv**.

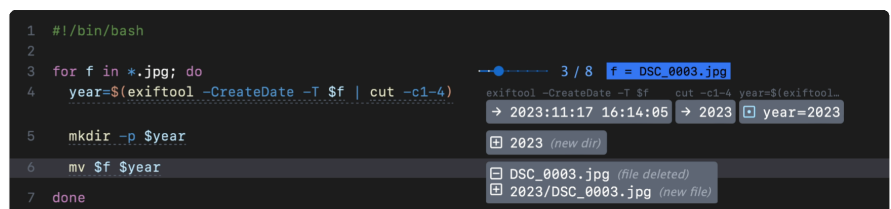


Figure 3: Using ShellVis to sort photos into folders by year. Screenshots show the visualizer interface (the right half of Figure 2).

bubble lets the user scroll its contents. They scroll past several irrelevant date fields to find one that looks correct. Scrubbing through iterations of the for loop, they confirm that the target date field changes as expected.

Step 3: Pipe & variable. To extract the year from the `exiftool` output, the programmer performs a number of steps. They pass some flags into `exiftool` to return just the “Create Date” field. Then they pipe this into `cut -c1-4` to return just the first four characters of this date – the year. Finally, they assign this result to the shell variable `year`. As they perform each of these changes, they save the file to see its effects in the visualizer. By the time they have finished this step, there are three bubbles visible: one for `exiftool`, one for `cut`, and one for the variable assignment. This script would not ordinarily print the outputs of `exiftool` and `cut`, as these are intercepted by ShellVis for display. The variable assignment bubble displays a different icon meaning “variable changed.”

Step 4: A mistaken move. Now the user tries to move the photos into directories named after years, with `mv $f $year`. The bubble on the right shows that the original file at `DSC_0004.jpg` is gone, and there is now a file at `2022`. This is a mistake! Since directories for years do not yet exist, `mv` interprets its arguments as asking to rename the photo to a file named after the year. In fact, a photo was moved to `2022` in an earlier iteration of the loop, and in this fourth iteration, it is being destructively overwritten with a second photo taken in 2022. Ordinarily, this mistake would be either inconvenient (if backups were made) or disastrous (if not). Fortunately, ShellVis sandboxes file system access: the user’s original files have not been touched and all writes to files are taking place in a simulated layer. The user can simply recognize the mistake and iterate on their script to fix it. Every execution of the script starts from the same state.

Step 5: Fixing the mistake. To fix their mistake, the user adds `mkdir $year` before `mv`. With this change, `mv`’s output bubble shows the move is performed successfully. The script is complete and the user can run it in their terminal.

5 INTERFACE DESIGN

In this section, we describe ShellVis’s design in more detail, focusing on how we augmented code with feedback. The following section goes into more detail on implementation, including sandboxing.

5.1 Scope

ShellVis is intended to bring live programming into shell programmers’ workflows in an incremental way while remaining compatible with existing practices.³ This led to several specific design decisions. One was to build ShellVis as a *within-code live programming environment* (§2.2). Here we discuss two other, related decisions.

First, we decided that ShellVis should support standard POSIX-family scripts (like `bash` and `zsh`), rather than using an alternative language or switching away from plain-text programming entirely. This decision was made with some reluctance. Compared to modern languages, the shell’s syntax and semantics are idiosyncratic,

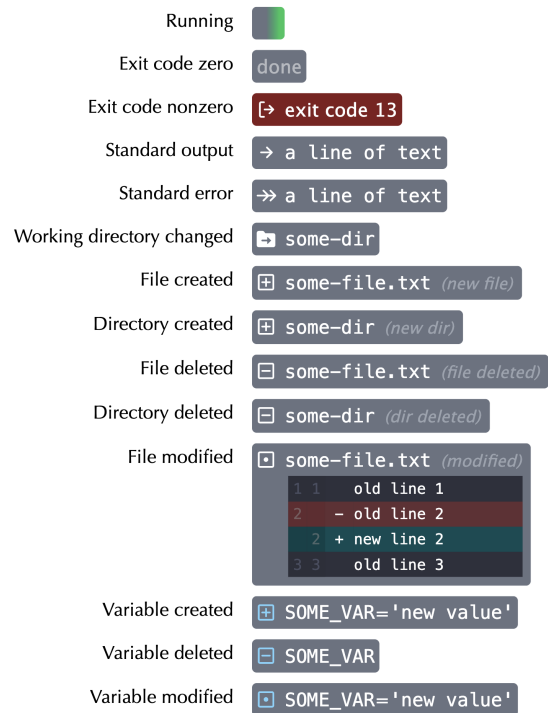


Figure 4: Annotations supported by ShellVis

and this seems to cause real problems for programmers. Many appealing alternatives are available.⁴ However, POSIX-family shell scripts are both technically and culturally entrenched. By building on them, ShellVis can be used with the enormous corpus of existing scripts and in the many contexts where shell scripts are necessary. Furthermore, from a research point of view, building a live-programming system on traditional shell scripts allows us to better understand the contribution of a live-programming interface without the complicating factor of a change in language, and it helps us maintain an approachable scope.

Second, we decided to make ShellVis work alongside any existing editor, rather than making our own editor or embedding ShellVis deeply inside a particular editor. For prototyping purposes, we could have built a custom editor on top of an open-source library like CodeMirror [20]. However, we felt that this would yield few benefits, and would have to be abandoned if we ever wanted to distribute ShellVis for actual use: existing editors are just too good, and programmers are just too attached to them. Embedding ShellVis deeply inside of an existing editor is an appealing idea. However, it would limit the reach of ShellVis to users that prefer that editor. Also, the extension APIs of editors are rarely general enough to support the particular visualizations and interactions we wanted ShellVis to have. This is a general problem: Projection Boxes [34] could not be implemented without forking the Visual Studio Code codebase, making distribution much more difficult.

³This goal of “incremental adoptability” fits under the “Sociability” dimension in the “Technical Dimensions of Programming Systems” classification [28].

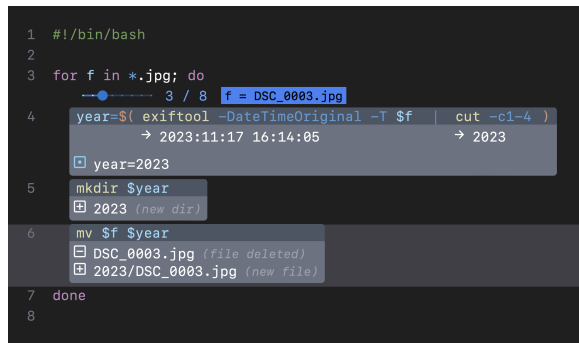


Figure 5: An earlier design for ShellVis which places bubbles “inline” with code, rather than off to the side.

5.2 Fisheye Information Bubbles

As ShellVis runs a shell script, it produces a program trace which it streams to the web browser visualizer. The visualizer displays the trace to the programmer, in the form of “information bubbles” attached to commands run by the script. Figure 4 shows the annotation bubbles currently supported by ShellVis. These annotations are all generic, in the sense that they show information that can apply to all commands. We believe they encompass most forms of feedback that shell-script authors need to see to understand the behavior of their scripts, with the exception of feedback about effects that ShellVis does not currently sandbox, like network access.

The ShellVis visualizer runs in a separate window from the code editor. Although this display is separate, it is essential that the programmer be able to shift their attention easily between their editor and the visualizer as they work. These shifts in attention become easier if the visualizer closely resembles code in the editor. (This is why, for instance, the visualizer matches Visual Studio Code’s syntax highlighting.) But, in tension to this goal, we often have many lines of information for each line of code, such as a large output text or multiple file system modifications. Showing all this information pushes successive lines of code down, resulting in a mismatch between the editor and visualizer. An earlier design for ShellVis, shown in Figure 5, placed feedback bubbles “inline” with commands in code rather than off to the side, and suffered from this mismatch problem.

Projection Boxes [34] faced a similar issue, and resolved it with per-line boxes that float to the right of the code. When the current line changes, the line’s box moves to be aligned with it, while boxes above and below get pushed out of the way, often off the screen. A limitation of this approach is that it becomes difficult to survey the behavior of an entire program at a glance when most lines’ boxes have been pushed entirely out of view. Put another way, Projection Boxes offer low peripheral visibility.

Our approach is instead to use “abbreviated bubbles” which expand to full bubbles when focused, a form of “fisheye view” [13, 14]. As shown in Figure 6, abbreviated bubbles are one or two lines in height and show a summary of the full bubble’s contents. Bubbles expand as “pop-ups” that may cover up bubbles lower on the page,

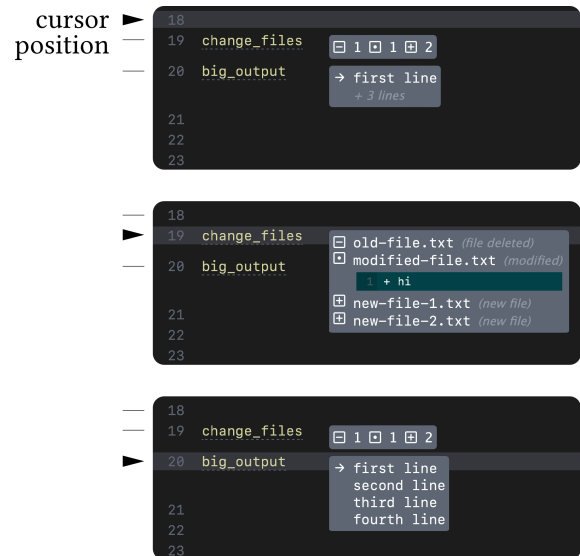


Figure 6: Examples of how bubbles expand and collapse as the editor’s cursor passes over them.

rather than move them. This helps to keep the overall visualization stable and predictable as focus changes.

As in Projection Boxes, visualization focus in ShellVis is generally controlled by moving the cursor in the editor. Since our visualizer is separate from the editor, a bit of machinery is needed to get cursor information from the editor to the visualizer. We made an extension for Visual Studio Code which reports this information to the ShellVis server via a WebSocket connection. Analogous extensions could be made for other editors. Bubbles can also be expanded in the visualizer by hovering over them with the mouse.

5.3 Visualizing Loops

Another design challenge is to visualize the execution of loops. Loops create a structural mismatch between a program’s source code and its trace: a single line of code in a loop may be executed many times. This presents a new instance of the problem above: how can this large amount of information be presented while maintaining a close correspondence with source code?

Early designs of ShellVis (shown in Figure 7) showed “expanded” views of loops, laying out loop iterations either horizontally or vertically, repeating the loop code for each iteration, and using the “inline” version of command bubbles shown in Figure 5. These designs appealed to us because they followed our general principle of showing as much information as possible by default, minimizing interaction and permitting comparisons across iterations with a mere movement of the eye.⁵ However, these designs disrupt the visual correspondence between the visualizer and source code in the editor, which is essential if the visualizer will be used in a tight loop with changes to code.

We ultimately opted to address this problem with interaction. As shown in Figure 3 and discussed in §4, ShellVis provides sliders for selecting iterations of interest inside for and while loops. As

⁴These include Xonsh [59], YSH [5], Nushell [44], Fish [11], and Murex [41].

⁵As Victor [58] says, “The hand is much slower than the eye”.

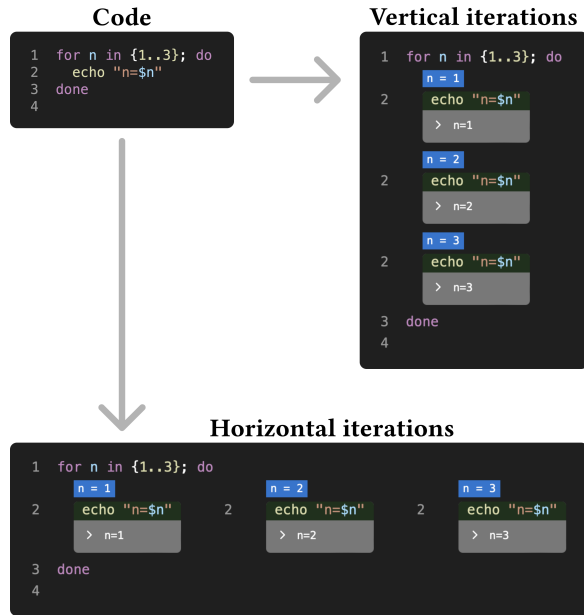


Figure 7: Two alternative designs for loops explored during ShellVis’s development which show all iterations simultaneously, rather than giving the user a slider to select one iteration at a time.

the user drags the slider, ShellVis shows either the value of the loop variable (for a for loop) or a command bubble for the loop condition (for a while loop). Annotations in the body of the loop update to reflect the selected iteration. Using the slider, the user can make sure the loop is behaving as expected across iterations.

5.4 Interface Scalability

The current UI design of ShellVis scales well to moderate amounts of data. If a loop has hundreds of iterations, the user can scrub through them as easily as if it had three. If a command has a large output, its output bubble will grow to a maximum size and then let the user scroll its contents. (One of our study tasks involves a file with several thousand lines.) We have not optimized our design or implementation for “big data”; within a shell context, handling hundreds to thousands of iterations or lines is sufficient for many end-user applications.

6 IMPLEMENTATION

ShellVis is implemented in two parts: a server and a client. The ShellVis server runs on the user’s own system. It is responsible for running a shell script in a sandboxed environment, tracing its activities, and reporting this information to the client as a live-updating program trace. The client runs inside a web browser. It is responsible for displaying the program trace to the user and allowing the user to explore it interactively.

Implementation of the ShellVis client was fairly routine. As a shell script executes and is traced, the trace is streamed from the server to the client with Automerge [2] and displayed with the

help of React [39]. The remainder of this section discusses the implementation of the ShellVis server.

6.1 Sandboxing

To make live programming of shell scripts possible, ShellVis must sandbox and trace the effects of scripts. Currently, sandboxing is limited to effects on the filesystem. ShellVis currently does nothing to sandbox other effects, such as network requests. We discuss the prospects for these other effects in §9.3.

ShellVis sandboxes the file system with a *union file system* [48]. With a union file system, a user can mount a virtual copy of the root file system inside a temporary directory (the “union directory”) such that reads go straight to the root file system but writes go to a “delta” directory which records file modifications. Subsequent reads of changed files route to the delta directory instead, making a union file system essentially a “copy-on-write” file system. If a shell script is run in the union directory rather than directly on the root file system, it can read the entire root file system, but any changes it performs will be recorded in the delta directory rather than touching real files. This protects the original file system.

The contents of the delta directory also tell us what file system modifications have occurred. As we wish to know more precisely what modifications are produced by *each line* of a script, ShellVis actually stacks two union file systems on top of one another. Each command is run in the upper union file system. After it is run, the upper delta directory is scanned for changes to add to the program trace, and the changes are committed to the lower union file system. The upper delta directory is then cleared for the next command.⁶

As mentioned in section §2.4, ShellVis’s approach to sandboxing is directly inspired by *try*, a command-line tool for Linux which lets users sandbox individual commands [56]. *try*’s sandbox is based on two Linux operating-system features: OverlayFS and namespaces. OverlayFS is a union file system built into Linux. Namespaces provide several important forms of process isolation. Most importantly for *try*, they make it possible to run a process with an arbitrary directory chosen as its root directory, so references to “/” in the program will resolve to that directory. *try* runs its target command in a namespace where “/” refers to the OverlayFS mount directory.

ShellVis is currently built for macOS, which does not have a built-in union file system. Instead, ShellVis uses unionfs-fuse [51], which is built in user space on top of FUSE. macOS also does not have namespaces. It does provide the more primitive *chroot* call, which provides the root-setting function of namespaces without its other features. While it should be enough for our purposes, we ran into several issues with *chroot*. Using *chroot* on macOS requires disabling System Integrity Protection, which is cumbersome and weakens the computer’s security. Even with SIP disabled, we were unable to make *chroot* work seamlessly on macOS. Without *chroot*, our sandboxing is leaky. File accesses performed with relative references are sandboxed, since they remain in the sandbox directory, but absolute paths will refer to original files outside the union file system. This disqualifies the current ShellVis prototype from being used in the wild by users who may be unaware of this

⁶Although this approach generally works, it may fail to ascribe changes to the correct command if multiple commands write to the file system concurrently. This could be fixed by using a more sophisticated virtual file system, or by using tracing tools which directly target individual processes like *dtrace* [4] or *fsatrace* [35].

limitation. Future work on ShellVis will likely move to Linux, where these problems can be more easily fixed and which is generally a more accommodating environment for OS-level experimentation.

6.2 Tracing

The ShellVis server watches a target script file for changes and re-runs the script whenever the editor saves a new copy. This and the selection monitoring mentioned in §5.2 are the only ways the editor communicates with ShellVis.

```
echo "hi!"

↓

local sv_ret >/dev/null
local sv_uploadId_stdout sv_uploadId_stderr sv_uploadId_varsEnter\
sv_uploadId_varsExit >/dev/null
sv_msg "{\type\": \"call-enter\", \"nodeId\": \"CallExpr_1_1_11\", \
  \"context\": \"$(sv_ctx_str)\", \"cwd\": \"$PWD\", \"suppressed\": false} | \
  read -r sv_uploadId_stdout sv_uploadId_stderr sv_uploadId_varsEnter\
  sv_uploadId_varsExit
sv_typeset | sv_upload $sv_uploadId_varsEnter
echo \"hi!\" \
  1>&1 1> >(sv_upload $sv_uploadId_stdout) 2>&2 2> \
  >(sv_upload $sv_uploadId_stderr)
sv_ret=$?
sv_typeset | sv_upload $sv_uploadId_varsExit
sv_msg "{\type\": \"call-exit\", \"nodeId\": \"CallExpr_1_1_11\", \
  \"context\": \"$(sv_ctx_str)\", \"cwd\": \"$PWD\", \"exitCode\": \"$sv_ret\"} | \
  >/dev/null
sv_exitcode $sv_ret
```

Figure 8: Example of how ShellVis transforms a minimal one-line shell script. The transformed script also includes a prelude defining the `sv_` functions, not shown here.

The server needs to somehow obtain detailed, line-by-line information about a shell script’s behavior when it runs. Our approach here was to transform the script before running it to add additional tracing instructions. An example of this transform is shown in Figure 8. The `sv_msg` and `sv_upload` functions are used in this transformed code to report events (like a command returning) and upload data (like standard IO streams from a command) to the ShellVis server over one-off HTTP connections. This is used for tracing, as well as to manage per-command sandboxing.

To perform the source transformation, ShellVis uses the `mvn/sh` library [36] to parse the script to an AST and then walks this AST to insert tracing code. If this parsing fails, the library often returns more useful error messages than the shell itself, which we report to the user. Currently, ShellVis instruments calls, for loops, and while loops. A future version of ShellVis could instrument more (conditionals, functions, etc.), but these were omitted in our prototype. A script with these features will run correctly in ShellVis, but with an incomplete trace.

An alternative approach to tracing a script’s execution would be to fork a shell’s codebase and add instrumentation directly to its interpreter, so an unmodified script can produce a trace when run. Through our implementation work, we have discovered that transforming a shell script without modifying its behavior can be a complex and error-prone endeavour, so instrumenting a shell directly may be preferable for future work.

6.3 Performance

ShellVis currently imposes a significant performance overhead over running a script directly. For instance, the final script constructed in section §4 takes around 3.5 seconds to run in ShellVis, and only 0.5 seconds if run directly. Profiling suggests that most of this overhead comes from communication between the instrumented shell script and the ShellVis server. Earlier in development, we explored more performant approaches to script-server communication, including named pipes and persistent TCP connections, but HTTP proved to be more reliable at this prototype stage. Moving the ShellVis server inside the shell (as described at the end of §6.2) would make lower-overhead communication easier to implement.

ShellVis relies on `unionfs-fuse` for sandboxing, so script commands that read or write to the file system must go through that system. We expect this incurs an additional cost. If ShellVis were moved to Linux, it would be able to use `OverlayFS`, which is likely faster since it runs in the kernel rather than in user space.

ShellVis’s approach of re-running a script from scratch whenever it changes will become less practical for larger or slower scripts, not due to overhead from ShellVis but due to the intrinsic cost of running the script. Consider scripts that process large media files like videos, or those that search through large directory structures. New techniques could make it more practical to live-program slower scripts like these. For instance, loops could automatically be limited to a smaller number of iterations for testing, or the results of long-running subcommands could be cached.

7 USER STUDY DESIGN

In order to assess whether sandboxed live-programming achieves its goals in the ShellVis prototype, and to provide further insight into the experiences of shell programmers, we conducted a user evaluation. Our study focused on three research questions:

- **RQ1:** What challenges do programmers face working with shell scripts?
- **RQ2:** How do programmers make use of live feedback when working with shell scripts?
- **RQ3:** How do programmers make use of sandboxing when working with shell scripts?

7.1 Participants

We recruited seven participants through social media channels. Shell-scripting experience ranged from 1 to 20 years. All our participants were men between the ages of 28 and 42.

7.2 Procedure

A study session began with a short interview focused on the participant’s history with shell-scripting and problems they ran into while working with shell scripts. We deliberately opened the session with this interview in order to obtain a participant’s prior perspective, unbiased by the approach of our prototype.

Next, we taught the participant how to use ShellVis through the photo-sorting scenario shown in §4. We ran ShellVis on our computer, accessed by the participant through Zoom’s remote control feature. We then asked participants to perform four tasks:

- **titles:** Rename text files to the titles contained in their first lines, with a possible pitfall caused by the presence of spaces in titles. Starts from a blank file.
- **users:** Delete user directories named in lines of a text file, with a possible pitfall caused by a blank line in the file. Starts from faulty code, such as might be produced by a code assistant.
- **log:** Read a complex pipeline which filters a log and aggregates information from it.
- **lines:** Multiply the line-counts of files in a folder.

Full prompts for these tasks are available in Appendix A. We encouraged participants to use Google or other resources naturalistically if needed, though we did not allow use of LLM assistants like Copilot and ChatGPT. (See §9.1 for some reflections on code assistants.) After completing these tasks, the participant filled out a survey, with questions and responses shown in Figure 9.

Finally, we conducted a post-study interview. This interview was not formally structured, but we focused on eliciting the participant’s impressions of ShellVis and how they thought it might or might not fit into their real-world workflows. In particular, we asked participants whether they thought ShellVis would address any challenges they mentioned at the start of the session, or not. Sessions ranged from one to two hours in length.

7.3 Limitations

Our study has significant limitations. The tasks we assigned to users were simple and designed to make use of ShellVis’s features. Our sample size is small and demographically narrow. While no participants were prior acquaintances of the authors, recruitment through social networks likely biased the study population towards people sympathetic to our goals. Since all participants had a least a year of shell experience, conclusions should not be drawn from this study about the value of ShellVis to shell-scripting novices.

8 USER STUDY RESULTS

To analyze the study data, the first author performed open coding on video recordings, akin to phases 1-2 of thematic analysis as described in Clarke & Braun [6]. We then sorted these under headings corresponding to our research questions and identified themes within each. The remainder of this section reports the results of this analysis. Quotes from participants are highlighted “like this” and labelled with participant IDs (like P₇).

8.1 Challenges with Shell Scripts (RQ1)

When we asked participants in our initial interview about problems they had run into with shell scripts, most participants were quick to recount difficulties. A few had pointedly negative feelings, responding that “everything” (P₂) or “essentially every part of [shell scripting]” (P₃) was challenging.

The most common problems mentioned were not knowing, or not remembering, how to do certain things in shell scripts. “Can you remember what the right thing to call is? Because there’s so many magic spells... [even] if you know the spell, how do you cast it?” (P₁). “I use [shell scripts] only frequently enough to remember that there is a way to do something, but I don’t remember exactly what is the way.” (P₃). “Every time, I reach for Google first, or now ChatGPT.”

	Disagree	Somewhat disagree	Neutral	Somewhat agree	Agree	Average score
Q1: ShellVis helped me write correct code.	0	0	0	1	6	4.9
Q2: ShellVis was easy to use.	0	0	0	2	5	4.7
Q3: ShellVis's line-by-line feedback was useful.	0	0	0	0	7	5.0
Q4: ShellVis's sandboxing of file changes was useful.	0	0	0	0	7	5.0
Q5: It was awkward that ShellVis put output in a browser window instead of the editor itself.	1	0	2	3	1	3.4
Q6: ShellVis could be useful in scripts I work with in my life.	0	0	0	1	6	4.9
Q7: ShellVis was fun to use.	0	0	0	0	7	5.0
Q8: I would use ShellVis again.	0	0	0	0	7	5.0

Figure 9: Questions in survey along with response distributions among the 7 participants. All questions are on a 5 point Likert scale with 1 being “Disagree” and 5 being “Agree”. Average scores are shown to the left.

(P₅). “[I]t’s a matter of, how do you bring up the right incantation to get it working right... I end up reaching for the manual [more] often than not...” (P₆). “I find some of the shell keywords quite challenging to remember... these things I constantly have to look up even though I write these time and time again.” (P₇). Participants often attributed these memory problems to not using shell scripts often. Shell scripts seem to occupy an unfortunate middle ground: used too infrequently to be remembered, but frequently enough that forgetting is a pain. Participants employ different techniques to get around these problems, including Google, ChatGPT, looking up examples of one’s prior work, and man pages. These “gulf-of-execution” struggles have a subtle relationship to ShellVis’s “gulf-of-evaluation” approach [43]. We discuss this issue further in §9.1.

Several participants reported they had run into disasters while working with the shell: “I’m extremely paranoid... because I have seen files disappear. And there’s nothing you can do about it later; it’s pretty hard to recover.” (P₃). To protect themselves against such disasters, shell-scripters employ different methods. P₃, P₄, and P₇ mentioned putting echo in front of potentially destructive commands to “dry run” them. Other participants tried to backup

files before starting work on a script. **P**₅ says “always make backups”, but also reported that backups are annoying to use, take up space, and you may simply forget to make them.

In the initial interview, participants did not generally mention lack of visibility, the problem live programming most directly addresses, as a problem. The one exception was **P**₇, who remarked: “As a shell script gets more and more complicated, there’s no observability into these pipelines... I think the lack of observability... is sometimes a little bit of a friction point.” Problems with lack of visibility were mentioned more often as participants worked through tasks with ShellVis and reflected afterwards.

8.2 Using Live Feedback (RQ2)

Participants unanimously agreed that ShellVis’s line-by-line feedback was helpful (Q3 avg of 5.0/5). As ShellVis is at its heart a visualizer, it is perhaps not surprising that participants found ShellVis useful for understanding code they hadn’t written (as specifically probed in **logs** and to a lesser extent **users**): “I think I kind of know what that regex does, but I’m not sure... [ShellVis] is nice because it gives you the output right there...” (**P**₁), “If I were looking into a shell script that I didn’t author myself, [ShellVis] would also be incredibly valuable... [Without ShellVis,] you only see the pieces of the pipeline, but not the data flowing through.” (**P**₇).

Participants also found ShellVis useful while actively writing code, to plan their next step or diagnose why their last step failed. **P**₇ explained this experience: “[With ShellVis,] when I’m reasoning about a future step, I can look at intermediate artifacts from previous steps without having to rerun. I could scan upward on the web view and be like: oh yeah, that variable I used earlier... it has this data, versus having to reload that context in my head after I had gone maybe two, three steps forward.” A specific example of this experience: While working on **titles**, **P**₄ wondered why a use of `$file` in their code wasn’t producing the expected result. They asked what value `$file` was taking on: “Can I see what `$file` is for each... Oh yeah, duh, it’s right there!”

Several participants described how live feedback provided confidence or comfort: “The preview [shows] that when you’re running this, the first file that gets picked up is this. That builds a certain confidence level within me that, oh, that’s right.” (**P**₆), “Just like knowing that things are successfully running is very comforting.” (**P**₇). We will discuss these affective dimensions more in §8.3.

8.2.1 ShellVis vs. echo Logging. A natural point of comparison for ShellVis is logging information from a script with `echo`. Participants agreed that ShellVis’s output annotations were a significant improvement over this status quo, and gave several reasons why.

First, automatic output annotations remove the tedium of adding and removing logs during a debugging process: “I no longer have to litter my [script] with echoes that I need to subsequently search for and clean up.” (**P**₇). In response to a question about echo-based logging, **P**₄ reported: “You can put all this crap in... and have it print all of it, but then you have to put in case handling so that it isn’t printing all the time, because you don’t want all this garbage debugging stuff going across your console.”

P₅ further noted that ShellVis’s “show everything” approach obviates the need to know *where* to place logs: “I think it’s huge because you’re removing a whole extra step... knowing where to

put the right echoes. I don’t have to think about that because it’s all there. [With logs] I wouldn’t really know which line is causing the problem. It could be a line further up that’s causing a downstream problem, and I could be putting the echoes in the wrong place.”

Finally, participants reported numerous ways it was more helpful to see output annotations in context, attached to specific lines of code and structures like loop iterations, than to see echoed output streaming in a terminal. **P**₃ reported that, with echo-based logging, “finding exactly where you are is difficult... if your logs are too sparse, mapping that back to the source can be difficult... Or if it’s in a for loop, at what execution did that happen?”. **P**₄ similarly reported that “It’s easy when you’re a couple of nested loops in to get lost on where you are. So to see [with ShellVis] what the machine thinks is very helpful.”

8.2.2 ShellVis vs. Debuggers. Debuggers did not arise in conversation as much as echo logging did, likely because debuggers are used less often by participants. **P**₄ had some experience with the PowerShell debugger, but explained that ShellVis was “a little bit [different], cause to see what variables are [with a debugger], you have to single-step stuff... Whereas this is like, let’s run it. And now I can just use the little slider thing to see as it was going along... It’s a good way to visualize what the shell script is doing.”

8.3 Using Sandboxing (RQ3)

Participants unanimously agreed that ShellVis’s sandboxing of changes to the file system was helpful (Q4 avg of 5.0/5). In our initial interviews (§8.1), participants described techniques they used in the past to protect themselves from accidents with the shell. After trying ShellVis, several explained how ShellVis removed the tedium and error-proneness of implementing those workaround techniques. **P**₄ described how they typically put lots of work into making “dry run” versions of scripts, and compared this to ShellVis: “If [ShellVis] was able to catch that kind of stuff as a sandboxing test and make the automated execution stop, that’s a pretty powerful aid – and something I usually have to manually add.” **P**₇ described their approach: “What I would ordinarily do is... pick one file to start and then set that as `$f`, so I might do something `f=/docs/doc1.txt`. And then... if one file does change destructively, I could easily roll back. Or alternatively, I might start off echoing everything and to be like, okay, that looks sensible. But with [ShellVis], I can just very confidently run the entire script, which is very cool.”

We were gratified to hear some participants using affective language like “confidence” and “comfort” when describing the experience of using ShellVis. **P**₆ said “[ShellVis] gives you a lot of confidence that, look, I can fix it before I actually run it. So I think the cognitive load... goes away. I’ll be less worried about executing this and making a mess of my system.” **P**₃ said “Having this safe environment makes me very comfortable with just wildly trying things and see what happens, versus I’m usually very, very cautious because deletions are very hard to undo.” This language of exploration and experimentation was echoed by **P**₇: “[ShellVis] is an environment that allows this kind of experimenting. You can kind of be aggressive.”

Affective dimensions like these may be an important (but difficult-to-measure) benefit of safe live-programming environments. Beyond making programming faster or less error-prone, these environments may cultivate the kind of play and experimentation that support learning and creativity. Our study participants universally reported that they had fun working with ShellVis (Q7 avg of 5.0/5).

Of course, a feeling of safety should be backed up by actual safety. Several participants made clear that ShellVis would need to earn their trust before they would feel comfortable using it (P₁, P₄). As discussed in §6.1, this wariness is warranted, as ShellVis’s file system sandboxing is currently incomplete.

8.4 Overall Impressions

After using ShellVis, participants were uniformly positive about its helpfulness in accomplishing study tasks (Q1 avg of 4.9/5) and its potential usefulness in their own work (Q6 avg of 4.9/5). All participants expressed an interest in using ShellVis in their own lives: “I think it’s really cool; I would love to have a tool like that” (P₁), “This is good enough that I actually would like to be able to run it on my computer” (P₂), “It already works amazingly well... It felt, like, transparent,” (P₃), “I would definitely use a tool like this” (P₄), “Is this coming out anytime soon?” (P₅), “Do you have an ETA for when this tool might potentially be made available?” (P₆), “I really want this tool and I would use it immediately.” (P₇).

9 DISCUSSION & FUTURE WORK

9.1 Gulf of Evaluation vs. Gulf of Execution

As discussed in §8.1, the most common problems participants reported from their shell-scripting experience had to do with not knowing how to express their intent as shell code, whether because they had never learned the shell’s syntax or because they forgot it. In the language of Norman & Draper [43], these problems come from the *gulf of execution*. At least at first glance, this seems quite unrelated to ShellVis’s approach. ShellVis is designed to help programmers understand what a shell script is doing, a clear instance of bridging the *gulf of evaluation* [43]. In other words: Once you type something, ShellVis can help you see what it does, but it can’t help you figure out what to type in the first place.

We asked participants about this in our final interview. They tended to agree with the interpretation above: “My initial response... is that I don’t think it helps there. Because to get the tool to do anything, you have to give it some input.” (P₁), “If I start off with the fact that I don’t know how to structure a for loop, then I don’t think [ShellVis] provides enough feedback to say, okay, this way.” (P₆). P₅ agreed, but added some nuance, suggesting that bridging the gulf of evaluation could in some situations assist in bridging the gap of execution: “Partially, maybe [ShellVis] helps because you see the error messages right there, and you can glean some information from the error messages... but it’s always better to know the commands beforehand.”

In a broader sense, it is clear from participant reports in §8.2 and §8.3 that ShellVis *did* help participants with the larger “execution” task of writing shell scripts, whether or not it was able to assist with low-level aspects like syntax and command flags.

LLM-powered code assistants, like Copilot [15] and ChatGPT [47], are an emerging class of systems intended to bridge the gap of

execution. While we did not test the use of ShellVis alongside code assistants, several participants expressed excitement that ShellVis could be a useful adjunct, especially with its sandboxing for extra protection. “I do think a common mode with [code assistants]... is like, Oh, great. I wrote this code. It looks right. Is it right? I don’t know. I guess I better check. And in that sense, I do think the the introspection afforded with this tool is pretty useful.” (P₂) “If [the code assistant] is wrong... with your tool I’ll see it immediately without destroying anything... So I would certainly use LLMs more if I’m using ShellVis, because mistakes won’t be costly.” (P₃) “Because I don’t trust code assistants, there’s still that verification phase... Because this tool provides me live feedback, it’s a very, very good way of verifying what the code assistant has put out. And the fact that it does it in a sandbox manner without a destructive mode is the key highlight.” (P₆) Further study is required to determine how well ShellVis supports the use of LLM-powered code assistants. We are encouraged by the results of Ferdowsi et al. [10], who find positive results testing the use of LLMs alongside Projection Boxes. Shell scripting is riskier than the contexts those researchers studied, so the need to validate LLM-generated code with (safe) live programming may be even greater.

9.2 System improvements

Interviews with participants surfaced numerous opportunities to improve ShellVis’s design and feature-set.

Editor integration. We were worried that our decision to run the ShellVis visualizer in a window separate from the editor (motivated in §5.1) would produce an awkward experience for users. Survey results partially support these concerns (Q5 avg of 3.4/5). P₂ and P₅ mentioned that the double-window arrangement took up valuable screen space they’d ordinarily use for other purposes, and P₂ was sometimes thrown off by the visual similarity of the two windows and tried to edit through the visualizer pane. Otherwise, this issue did not arise much in discussion, though that may be a product of the laboratory setting. A direction to explore in future work would be integrating ShellVis’s line-by-line annotations into an existing text editor so that a second window is unnecessary. As discussed in §5.1, we deliberately chose not to do this for our first prototype, but ShellVis’s client-server architecture should make further experiments with editor integrations straightforward.

Surveying across loop iterations. Several participants noted that, though the loop scrubbing interaction was easy to use, it meant that anomalous behavior could hide in iterations that weren’t visible at the moment. “Even though the for-loop scrubbing was cool, it would be nice to be able to see each loop in aggregate... If I could just quickly scan through a list and see all loop iterations in some kind of way that grouped each iteration, I think that [spotting an anomaly] would have also been really easy...” (P₇). Different approaches may accommodate the need to *survey* many iterations of a loop at once. For instance, “unfolded” designs like those shown in Figure 7 or grid views like those used in Projection Boxes [34] could show annotations for all iterations simultaneously. Alternatively, the existing slider could be maintained, but sparklines like those used in Hoffswell et al. [23] could be added to provide just enough “peripheral vision” for anomalies to be spotted from afar.

Connecting lines across commands. While the loop view in ShellVis makes it easy to see chains of cause and effect within an iteration, idiomatic shell usage often involves commands that process streams line-by-line without loops, like `sed` and `awk`. (Our **log** task highlighted commands like this.) When line-by-line commands are piped together, ShellVis shows the outputs of each stage of the pipeline in separate bubbles, distancing corresponding input & output lines. **P₃** and **P₇** both noted that this made it hard to follow the transformation of lines by commands, and they wanted a way to “go from a line output of a command line program and trace it back to an earlier command’s line” (**P₇**). This desire makes perfect sense from a user’s perspective, but it may be challenging to implement because Unix commands do not expose information on mappings between their input and output lines, and these relationships vary between tools.

Extensible visualizers and editors. While ShellVis’s generic annotation bubbles are usually enough to show what a shell script is doing, domain-specific visualizations for particular commands or situations could go even further. For instance, **P₅** often writes scripts that process JSON data which they need to inspect. They currently use Postman [49] for this, which provides a better interface than the shell for looking at data, but which requires moving away from their script into another interface. As another example, **P₇** mentioned wanting to color-code log output to more easily spot patterns in a time series, and to visualize 3D point clouds processed by another script. These examples suggest the potential of a system whereby ShellVis’s visualizations could be extended for specific commands or output types. As a lighter-weight alternative to a de novo extension system, **P₇** proposed that shell scripts could make their own visualizations by generating HTML which ShellVis would display in rendered form. Beyond visualizing output, extensions to ShellVis could change how parts of a script are *authored*. **P₁** suggested “a GUI for any command line – as long as there’s a tool that someone likes, or if you’re a developer of a tool, you could write some sort of GUI spec that plugs into ShellVis”. **P₇** imagined a system where the programmer could directly edit a command’s output and the system could infer a change to the command to match the desired manipulation—a vision similar to output-directed programming [22]. Systems like *livelits* [46] and *Engraft* [25] embed live GUIs inside of code, suggesting pragmatic ways to achieve these “richer” visions [26].

More shell features. ShellVis does not yet provide feedback for the full gamut of shell features, as mentioned in §6.2. The tasks we asked participants to perform did not call for the features ShellVis does not support, so these limitations did not arise much in our interviews. The one limitation noted by a study participant was when **P₂** was working on **titles**: “It would be cool to see not only the command I’m running [in the source file], but also what actually gets run in the post interpolation step.” We agree, as this feature would help illuminate many mistakes from misuse of the shell’s idiosyncratic interpolation and quotation features.

Remote use. At least three participants (**P₂**, **P₃**, **P₄**) wondered if ShellVis could be used to edit scripts running on remote systems. While ShellVis does not currently support this, its client-server architecture should make it easy to adapt to this use: the server can run on a remote system while the client runs locally in a user’s browser.

9.3 Sandboxing More Effects

Not all side effects of shell scripts take place through the file system. The second-most prominent side effect of scripts is probably network requests. A network request can be patently side-effectful, such as a request to a database server to delete a user’s account. Even seemingly innocuous requests, such as to HTTP GET endpoints, might cause problems if made indiscriminately, like costing money for a paid API or triggering rate limits. Beyond the file system and network requests, there are many other ways scripts can cause effects: writing to the clipboard, modifying running processes, making system calls, or causing physical effects like a print job.

These effects call for different approaches to sandboxing. We might call our current file system approach *simulation*, as we are able to, with high fidelity, simulate a script’s effects on the file system using a union file system. In contrast, it is impossible to simulate network requests to arbitrary hosts, as we cannot anticipate, in general, how remote hosts might respond to requests.

When simulation is not possible, the simplest alternative approach is *detecting and quarantining* side effects. This involves detecting that a command will have an effect, stopping execution, showing the user the intended effect (in the context of the program trace), and giving them options of how to proceed. If the effect is entirely unexpected and unwanted, they can stop execution and revise the program to fix the problem. If they deem the effect harmless, they can approve it and execution will continue. If the effect looks like the right effect to perform, but the user doesn’t want to perform it yet due to irreversible side-effects (say, on a remote host), they can mock it: provide, by hand, what they anticipate the response will be so that execution can continue. On top of the manual interactions described above, layers of automation are possible: commands could be placed on allowlists and denylists, or mock utilities could be provided to automate the mocking of a command.

We have not tested any of these approaches yet. Encouragingly, **P₄** (a system administrator who frequently writes “potentially destructive” scripts) brought up some of these ideas themselves in our conversation: “If there was a way to have the tool say, okay, I would have run this, we’re going to assume that it worked and set the variable to this predefined thing so that the script execution can continue.” **P₄** was the only participant who brought up the need to sandbox more than the file system. This should not be taken as a sign that the issue is unimportant; we expect our study design did not prime participants to think about this issue.

9.4 Sandboxed Live Programming Beyond Shell Scripts

We believe the scope of the *sandboxed live programming* technique is much broader than just shell scripts, as practical situations typically call for side-effects. The need for sandboxing may be especially strong in end-user programming contexts, both because end users’ concerns are often tightly tied to immediate effects they wish to perform, and because end users often have less experience programming and require more support.

How might the sandboxing approach of ShellVis translate from the Unix world of files and command-line programs to the GUI world? For instance, AppleScript [7] and Shortcuts [1] are programming systems for automating graphical applications. What would

it take to sandbox these application-centered programming systems? Could it enable a higher degree of live feedback, as it did for ShellVis?

Beyond familiar computing environments, new possibilities on the horizon may enable richer forms of sandboxing. Local-first software [32] reproduces the conveniences of modern cloud computing while allowing transparent local access to data. CRDTs, the data structure underlying many local-first systems, innately support branching and merging, capabilities which could be used to sandbox scripts. Local-first software may be a fertile substrate for broader use of sandboxed live programming.

9.5 An Environment for Learning

Our study participants were proficient programmers. We don't expect that someone without shell-scripting experience would perform well (or have a good time) in our study, especially given how little ShellVis does to help bridge the gap of execution (§9.1). Nevertheless, we hope that ShellVis might make shell scripting more accessible to novices, as an adjunct to other learning resources. Live-programming environments have previously shown promise in learning contexts [27, 30]. We speculate that, by removing the risk of an irreversible mistake, ShellVis's sandboxing could make it easier for novices to learn through experimentation. P₄ joined our speculations: "[ShellVis] would make scripting a lot more accessible to people, I think, because you can still build iteratively, but now it's easier to see what each iteration would be doing."

10 CONCLUSION

The reach of live programming has been limited by its incompatibility with side effects. We have presented a general approach to this problem, *sandboxed live programming*, and tested the approach through *ShellVis*, our prototype of sandboxed live shell scripting. *ShellVis* combines overlay file systems with focus+context visualizations to create a new way of interacting with the shell, with the immediacy of the command line, the rich feedback of a notebook, and the program-editing capabilities of an editor. Participants in our study found the live visibility enabled by sandboxing helpful in understanding and writing code, and they enjoyed the feeling of safety provided by sandboxing. Discussion with participants also revealed the importance of complementing *ShellVis* with other tools to completely bridge the gulf of execution. With the development of further forms of sandboxing, these learnings can be extended to help make live programming practical in diverse end-user contexts with side effects.

REFERENCES

- [1] Apple Inc. 2024. Shortcuts User Guide - Apple Support. <https://support.apple.com/guide/shortcuts/welcome/ios>.
- [2] Automerge contributors. 2024. Automerge CRDT. <https://automerge.org/>.
- [3] Rocky Bernstein. 2024. Pipecut. <https://bashdb.sourceforge.net/>.
- [4] Bryan M Cantrill, Michael W Shapiro, and Adam H Leventhal. 2004. Dynamic Instrumentation of Production Systems. In *Proceedings of the General Track: 2004 Usenix Annual Technical Conference*.
- [5] Andy Chu. 2024. Oils. <https://www.oilshell.org/>.
- [6] Victoria Clarke and Virginia Braun. 2016. Thematic analysis. *The Journal of Positive Psychology* 12, 3 (dec 9 2016), 297–298.
- [7] William R. Cook. 2007. AppleScript. In *Proceedings of the third ACM SIGPLAN conference on History of programming languages*. ACM.
- [8] Mateusz Czapliński. 2021. up - the Ultimate Plumber. <https://github.com/akavel/up>.
- [9] Loris D'Antoni, Rishabh Singh, and Michael Vaughn. 2017. NoFAQ: synthesizing command repairs from examples. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. ACM, 582–592.
- [10] Kasra Ferdowsi, Ruanqianqian (Lisa) Huang, Michael B. James, Nadia Polikarpova, and Sorin Lerner. 2024. Validating AI-Generated Code with Live Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–8.
- [11] fish-shell contributors. 2024. fish shell. <https://fishshell.com/>.
- [12] Free Software Foundation, Inc. 2020. Bash - GNU Project. <https://www.gnu.org/software/bash/>.
- [13] G. W. Furnas. 1986. Generalized fisheye views. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 16–23.
- [14] George W. Furnas. 2006. A fisheye follow-up. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 999–1008.
- [15] GitHub, Inc. 2024. GitHub Copilot · Your AI pair programmer. <https://github.com/features/copilot>.
- [16] Michael Greenberg and Austin J. Blatt. 2019. Executable formal semantics for the POSIX shell. *Proceedings of the ACM on Programming Languages* 4 (dec 20 2019), 1–30.
- [17] Michael Greenberg, Konstantinos Kallas, and Nikos Vasilakis. 2021. The future of the shell. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. ACM, 240–241.
- [18] Michael Greenberg, Konstantinos Kallas, and Nikos Vasilakis. 2021. Unix shell programming. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. ACM, 104–111.
- [19] Michael R. Greenberg, Konstantinos Kallas, Nikos Vasilakis, and Stephen Kell. 2021. Report on the "The Future of the Shell" Panel at HotOS 2021. <https://api.semanticscholar.org/CorpusID:237605419>. ArXiv abs/2109.11016 (2021).
- [20] Marijn Haverbeke. 2024. CodeMirror. <https://codemirror.net/>.
- [21] Kieran Healy. 2024. Modern Plain Text Computing. <https://mptc.io/>.
- [22] Brian Hempel, Justin Lubin, and Ravi Chugh. 2019. Sketch-n-Sketch. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*. ACM, 281–292.
- [23] Jane Hoffswell, Arvind Satyanarayan, and Jeffrey Heer. 2018. Augmenting Code with In Situ Visualizations to Aid Program Understanding. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, 1–12.
- [24] Vidar Holen. 2024. ShellCheck - shell script analysis tool. <https://www.shellcheck.net/>.
- [25] Joshua Horowitz and Jeffrey Heer. 2023. Engraff: An API for Live, Rich, and Composable Programming. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–18.
- [26] Joshua Horowitz and Jeffrey Heer. 2023. Live, Rich, and Composable: Qualities for Programming Beyond Static Text. (2023). Presented at the 13th annual workshop on the intersection of HCI and PL (PLATEAU 2023).
- [27] Ruanqianqian (Lisa) Huang, Kasra Ferdowsi, Ana Selvaraj, Adalbert Gerald Soosai Raj, and Sorin Lerner. 2022. Investigating the Impact of Using a Live Programming Environment in a CS1 Course. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*. ACM, 495–501.
- [28] Joel Jakubovic, Jonathan Edwards, and Tomas Petricek. 2023. Technical Dimensions of Programming Systems. *The Art, Science, and Engineering of Programming* 7, 3 (feb 15 2023).
- [29] Idan Kamara. 2024. explainshell.com - match command-line arguments to their help text. <https://www.explainshell.com/>.
- [30] Hyeonsu Kang and Philip J. Guo. 2017. Omnicode. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*. ACM, 737–745.
- [31] Saketh Kasibatla and Alex Warth. 2017. Seymour: Live Programming for the Classroom. (2017). Presented at the Workshop on Live Programming (LIVE) 2017.
- [32] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-first software: you own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 154–178.
- [33] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussanier, Jonathan Frederic, Kyle Kelley, Jessica B. Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, Carol Willing, and Jupyter Development Team. 2016. Jupyter Notebooks - a publishing format for reproducible computational workflows. In *International Conference on Electronic Publishing*.
- [34] Sorin Lerner. 2020. Projection Boxes: On-the-fly Reconfigurable Visualization for Live Programming. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–7.
- [35] Jorge Acereda Maciá. 2024. Filesystem Access Tracer. <https://github.com/jacereda/fsatrace>.
- [36] Daniel Marti. 2024. mvdan/sh. <https://github.com/mvdan/sh>.
- [37] David W. Maxwell. 2015. Pipecut Project Home. <https://web.archive.org/web/20220409090300/pipecut.org>.

- [38] Susan McGregor. 2024. Adding Headers and Using Shell Scripts. <https://data.journalism.columbia.edu/content/adding-headers-and-using-shell-scripts>.
- [39] Meta Platforms, Inc. 2024. React. <https://react.dev/>.
- [40] Microsoft Corporation. 2024. How to Debug Scripts in Windows PowerShell ISE. <https://learn.microsoft.com/en-us/powershell/scripting/windows-powershell/ise/how-to-debug-scripts-in-windows-powershell-ise>.
- [41] Laurence Morgan. 2024. Murex. <https://murex.rocks/>.
- [42] Bonnie A Nardi. 1993. *A small matter of programming*. MIT Press, London, England.
- [43] Donald A. Norman and Stephen W. Draper. 1986. *User Centered System Design*. CRC Press.
- [44] Nushell Project Developers. 2024. Nushell. <https://www.nushell.sh/>.
- [45] Observable Inc. 2022. Observable - Explore, analyze, and explain data. As a team. <https://observablehq.com/>.
- [46] Cyrus Omar, David Moon, Andrew Blinn, Ian Voysey, Nick Collins, and Ravi Chugh. 2021. Filling typed holes with live GUIs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, 511–525.
- [47] OpenAI, L.L.C. 2024. ChatGPT. <https://chatgpt.com/>.
- [48] Jan-Simon Pendry and Marshall Kirk McKusick. 1995. Union Mounts in 4.4 BSD-Lite. In *USENIX 1995 Technical Conference*.
- [49] Postman, Inc. 2024. Postman API Platform. <https://www.postman.com/>.
- [50] Guillaume Pothier, Éric Tanter, and José Piquer. 2007. Scalable omniscient debugging. *ACM SIGPLAN Notices* 42, 10 (oct 21 2007), 535–552.
- [51] Bernd Schubert Radek Podgorny. 2024. unionfs-fuse. <https://github.com/rpodgorny/unionfs-fuse>.
- [52] David Rauch, Patrick Rein, Stefan Ramson, Jens Lincke, and Robert Hirschfeld. 2019. Babylonian-style Programming: Design and Implementation of an Integration of Live Examples into General-purpose Source Code. *The Art, Science, and Engineering of Programming* 3, 3 (feb 1 2019).
- [53] Dennis M. Ritchie. 1980. *The evolution of the unix time-sharing system*. Springer Berlin Heidelberg, 25–35.
- [54] Dennis M. Ritchie and Ken Thompson. 1974. The UNIX time-sharing system. *Commun. ACM* 17, 7 (7 1974), 365–375.
- [55] Matúš Sulir, Sergej Chodarev, and Milan Nosál. 2023. Outside the Sandbox: A Study of Input/Output Methods in Java. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. ACM, 253–258.
- [56] The PaSh Authors. 2024. binpash/try. <https://github.com/binpash/try>.
- [57] University of Cambridge Information Services. 2024. Unix: Simple Shell Scripting for Scientists. <https://training.cam.ac.uk/ucs/course/ucs-scriptsci>.
- [58] Bret Victor. 2006. Magic Ink: Information Software and the Graphical Interface. <https://worrydream.com/MagicInk/>.
- [59] xonsh developers. 2024. The Xonsh Shell — Python-powered shell. <https://xon.sh/>.

A STUDY TASKS

Below are the written prompts for the four tasks in our study, including any starter code that was provided.

A.1 titles

The “docs” directory contains a few speeches. Each lists its title as its first line. Write a script that renames each file to its title.

(Participants discovered that some of the titles contained spaces, which could cause trouble unless they were appropriately quoted.)

A.2 users

We’re running a web service, and need to respond to some users’ requests to delete their accounts. We have a list of usernames in a file called `users-to-delete.txt`. Write a script that deletes the user folders for each user in that file.

A code assistant generated the following code. Is it correct?

```
cat users-to-delete.txt | while read LINE; do
    rm -rf "users/$LINE"
done
```

(Participants discovered that `users-to-delete.txt` contained a spurious blank line, which the above script incorrectly interprets as a command to delete the entire `users` folder.)

A.3 log

What does this script do?

```
cat launchd.log |
  egrep 'Sandbox restriction$' |
  sed -n 's/.*requestor = \(.*\)\[.*/\1/p' |
  sort |
  uniq -c
```

A.4 lines

There are some files in the `files` directory. Write a script that computes the product of their line counts. (Ex: If there were two files, one with 10 lines and one with 20, the product would be 200.)

The starter code below counts lines, but doesn’t do any multiplying.

```
product=1
for f in files/*; do
    wc -l < $f
done
```